Facultad de Ingeniería UNMDP
Calidad de Software A / Taller de Programación A

Estrategia de Testing Aplicación: Clínica

Trabajo práctico final

Grupo 1

Arriaga Mario Esteban
Casanovas Di Sera Mia
Laz Octavio
Seneca Gabriel

Fecha de entrega: 11/11/2026

Índice

Introducción **1**
Test de AsociadoDTO 2
¿Qué se testeó? 2
¿Cómo se testeó? 2
Error encontrado 3
Test de integración 3
¿Qué se testeó? 3
¿Cómo se testeó? 4
Test del patrón Singleton 4
Test de la prueba CRUD 4
Errores encontrados 4
Test de integración con Mockito **5**
¿Qué se testeó? 5
¿Cómo se testeó? 5
Errores encontrados 6
Apéndice A **7**
Test caja negra 7
Definición de escenario general 7
public void SetNombre(String nombre) 7
public void SetApellido(String apellido) 7
public void SetDni(String dni) 8
public void SetCiudad(String ciudad) 8
public void SetCalle(String calle) 9
public void SetTelefono(String telefono) 9
public void SetNumero(int numero) 10
Resultados Obtenidos y Análisis de Inconsistencias 11
Apéndice B **13**
Definición de escenarios 13
Tablas de Partición y Baterías de Pruebas 13
Public static BaseDeDatos getInstancia() 13
Public void iniciarBD() 14
Public void guardar(Asociado asociado) 15
Public ArrayList<AsociadoDTO> cargarConLimite(int limite) 16
Public AsociadoDTO cargar(int id) 17
Public void actualizar(Asociado asociado) 17
Public void eliminar(String dni) 18
Resultados obtenidos 19
Verificación de la conexión del patrón Singleton 21
TestCargarIndividualPorId() 21
Resultado de la prueba del crud 25
Apéndice C **27**
Definición Formal de Escenarios 27

Tablas de Particiones y Baterías de Pruebas 27
Public void solicitarMantenimiento(Operario o) 27
Public void atenderDomicilio(Asociado a) 28
Public void trasladarALaClinica(Asociado a) 29
Public void regresarSinPaciente() 29
Rutina Concurrente: RetornoAutomatico.run() 30
Recurso Compartido y Concurrencia: Ambulancia (setOcupado / isOcupado) 30
Resultados Obtenidos y Análisis de Inconsistencias 31

Introducción

El presente informe documenta la estrategia de testing aplicada sobre la aplicación de gestión de una clínica, desarrollada en Java. El objetivo del trabajo fue verificar el correcto funcionamiento de distintos componentes del sistema, detectar defectos en el código provisto y proponer las correcciones correspondientes.

Las pruebas se organizaron en tres etapas, cada una con un nivel de integración creciente:

1. Pruebas unitarias de la clase AsociadoDTO, verificando sus métodos de acceso y modificación de forma aislada.
2. Pruebas de integración entre AsociadoDTO, DataAccessObject y BaseDeDatos contra un motor MySQL real, evaluando el patrón Singleton, la inicialización de la base de datos y las operaciones CRUD.
3. Pruebas de la clase Ambulancia y su máquina de estados, utilizando dobles de prueba (Mockito) para aislar su comportamiento y pruebas de concurrencia para evaluar su interacción con hilos reales.

Para el diseño de los casos de prueba se aplicaron técnicas de caja negra, principalmente particiones de equivalencia y análisis de valores límite, y se utilizó JUnit 5 como framework de ejecución. En cada etapa se registraron los resultados obtenidos, los errores encontrados y las soluciones aplicadas.

Test de AsociadoDTO

¿Qué se testeó?

Se implementó una suite de pruebas unitarias para verificar el correcto funcionamiento de los métodos setters y getters de la clase “AsociadoDTO”. El objetivo fue comprobar que los atributos del objeto se asignen y recuperen correctamente, así como también verificar que se respeten las validaciones de los datos ingresados.

Las pruebas se realizaron de manera aislada, sin necesidad de utilizar bases de datos ni otros servicios externos. De esta forma, fue posible evaluar el comportamiento de la clase y comprobar cómo responde ante valores válidos e inválidos.

¿Cómo se testeó?

Se aplicó un criterio de selección de valores de prueba. Para seleccionarlos, se utilizaron técnicas de caja negra, principalmente particiones de equivalencia y análisis de valores límite. Estos métodos permiten evaluar diferentes comportamientos del sistema a partir de la elección de valores representativos de cada caso.

- Valor ‘-1’: se utiliza para comprobar si el sistema rechaza los números negativos cuando el rango permitido comienza en cero.
- Valor ‘0’: se eligió para verificar que se acepte correctamente el límite inferior del rango válido.
- Valor ‘1’: se utilizó como ejemplo de un valor válido, con el fin de comprobar el funcionamiento habitual de los métodos.
- Valores ‘null’: se emplearon para verificar el comportamiento de los atributos de texto cuando no reciben un valor y comprobar si se aplican correctamente las restricciones correspondientes.

Error encontrado

Se encontró un error en el valor límite 0 del método setNumero(int numero). Según el contrato, el número de calle debe ser mayor o igual a 0, por lo que 0 es un valor válido y debería aceptarse.

Sin embargo, el código provisto validaba el dato con la aserción assert numero > 0, que excluye al 0. Al ejecutar la prueba testSetNumeroValidoLimiteInferior() con ese valor, el método lanzó un AssertionError y la prueba falló. La validación del código no coincidía con lo que pide el contrato.

Para corregirlo, se reemplazó la aserción por assert numero >= 0, que sí respeta el contrato. Con esta corrección, todas las pruebas de AsociadoDTO pasaron en verde, y se pudo continuar con el test de integración.

El detalle de las pruebas y las capturas de ambas ejecuciones se encuentran en el Apéndice A.

Test de integración

¿Qué se testeó?

En esta etapa se realizaron pruebas de integración para comprobar el funcionamiento conjunto de los componentes “AsociadoDTO”, “DataAccessObject” y “BaseDeDatos” con el motor de base de datos MySQL.

El objetivo fue verificar que los componentes pudieran intercambiar información de forma correcta, y que las operaciones de acceso y manipulación de datos se ejecuten de acuerdo a lo esperado. A diferencia de las pruebas unitarias, que evalúan cada componente por separado, estas pruebas permitieron analizar la interacción entre las distintas partes del sistema.

¿Cómo se testeó?

Para realizar las pruebas, se ejecutaron distintos casos de prueba destinados a verificar la comunicación entre los componentes y su interacción con la base de datos.

- Preparación del entorno: Se verificó que la conexión con MySQL estuviera disponible, para luego evaluar la comunicación entre objetos en memoria y el almacenamiento persistente, verificando así el ciclo de vida de la conexión bajo el patrón Singleton y la regeneración del esquema con sus datos semilla.
- Pruebas de operaciones CRUD: Se evaluó el comportamiento de las operaciones de inserción, consulta, modificación y eliminación de registros, comprobando que los cambios se reflejan directamente en la base de datos.
- Manejo de errores: se analizó cómo respondía el sistema ante posibles errores durante la ejecución de las consultas y el procesamiento de la información.
- Verificación de los datos: se comprobó que la información recuperada de la base de datos se asocia correctamente con los atributos de los objetos `AsociadoDTO`.

Test del patrón Singleton

Se evaluó la correcta implementación y comportamiento del Patrón de Diseño Singleton aplicado en la clase BaseDeDatos. El objetivo fundamental fue comprobar dos aspectos críticos de la arquitectura de persistencia:La integridad de la conexión inicial: Verificar que al invocar por primera vez la obtención de la instancia, el objeto de conexión (conexión) no sea nulo, garantizando que el sistema está listo para comunicarse con el motor relacional; la unicidad de la instancia: Comprobar que llamadas sucesivas al método de creación (getInstancia()) devuelvan exactamente la misma referencia en memoria, evitando la creación de múltiples conexiones redundantes al servidor MySQL.

Test de la prueba CRUD

Se probaron las cuatro operaciones del DataAccessObject contra MySQL. Antes de cada prueba se reinició la tabla con los 20 asociados semilla, para que todas partan del mismo estado.

Errores encontrados

Sobre el código provisto, las pruebas detectaron dos errores. La consulta de un asociado individual no funcionaba debido a la falta de un .next(), y la modificación siempre actuaba sobre el mismo registro, sin importar cuál se quería cambiar. Las altas, las bajas y las validaciones de datos funcionaron correctamente desde el principio. Una vez corregidos los dos errores, todas las operaciones pasaron las pruebas.

Además se encontraron dos inconsistencias con el SRS:

- actualizar() busca el registro por id y no por DNI, como pide el SRS.
- eliminar() deja pasar un DNI null.

Las pruebas de integración detectaron errores que solo aparecen al comunicarse con la base de datos real. Una vez corregidos, el Singleton, la carga de datos semilla y las operaciones CRUD funcionan correctamente.

El análisis detallado de cada error, las capturas de código y las soluciones aplicadas se encuentran en el Apéndice B al final de este documento.

Test de integración con Mockito

¿Qué se testeó?

Se evaluó el comportamiento del módulo que gestiona la ambulancia, el subsistema de emergencias, analizando dos aspectos fundamentales: por un lado, la delegación atómica de transiciones desde la clase Ambulancia hacia su máquina de estados estructurada bajo la interfaz State (solicitarMantenimiento, atenderDomicilio, trasladarALaClinica y regresarSinPaciente); por otro lado, la interacción con procesos concurrentes independientes, verificando el ciclo de vida continuo de la rutina en segundo plano RetornoAutomatico y la liberación de la ambulancia como recurso compartido, de modo que los hilos que esperan por ella puedan ser atendidos.

¿Cómo se testeó?

Para aislar la clase Ambulancia sin depender de las implementaciones concretas de los estados (DisponibleState, EnTallerState, etc.), se aplicó una estrategia de integración descendente (top-down) utilizando el framework Mockito en conjunto con JUnit 5:

- Dobles de prueba (Mocks): Se generó un objeto simulado de la interfaz State y de sus colaboradores (Operario, Asociado), inyectándolo en la ambulancia para inspeccionar invocaciones directas.

- Verificación de comportamiento: Se utilizó “verify” para verificar que cada acción ejecutada en la ambulancia delegue el método exacto correspondiente en la máquina de estados, y “doThrow(...)” para comprobar que las excepciones generadas en un estado se propaguen limpiamente hacia las capas superiores.

- Pruebas de concurrencia: Se evaluó la ejecución en paralelo instanciando RetornoAutomatico en un hilo independiente para verificar que la rutina invoque periódicamente el retorno a la clínica (CP-CONC-01), y se dejó un hilo bloqueado esperando la ambulancia ocupada para verificar que, al producirse el retorno automático, la ambulancia sea liberada y el hilo en espera sea atendido (CP-CONC-02).

Errores encontrados

Se ejecutó la suite AmbulanciaMockConcurrenciaTest (7 pruebas). Las 5 pruebas con mocks pasaron, lo que confirma que Ambulancia delega correctamente cada solicitud en la interfaz State. Las 2 pruebas de concurrencia fallaron y revelaron dos defectos del código.

CP-CONC-01: el retorno automático nunca se ejecuta.

La prueba falló con WantedButNotInvoked: RetornoClinica(). El hilo RetornoAutomatico debería devolver la ambulancia a la clínica periódicamente mientras la simulación está activa, pero la condición de su bucle está invertida. Con la simulación activa, el hilo termina apenas arranca y nunca invoca el retorno.

CP-CONC-02: el retorno automático no libera la ambulancia.

La prueba falló con WantedButNotInvoked: SolicitudDeAtencionDomicilio(). Con la ambulancia ocupada, un hilo que pide atención a domicilio queda esperando. Al ejecutarse retornoAutomatico(), el método despierta a los hilos en espera, pero no marca la ambulancia como libre. El hilo despierta, ve que la ambulancia sigue ocupada y vuelve a esperar, por lo que nunca es atendido.

- Solución aplicada: Se agregó la llamada a setOcupado(false) antes de notifyAll() en retornoAutomatico(), de modo que los hilos en espera encuentren la ambulancia libre y puedan ser atendidos.

Apéndice A

Test caja negra

Definición de escenario general

Nro de escenario	Descripción
1	Se dispone en memoria de una instancia de la clase AsociadoDTO creada mediante su constructor por defecto (new AsociadoDTO()), con sus atributos en estado inicial por omisión, lista para la invocación aislada de sus métodos inspectores y modificadores.

public void SetNombre(String nombre)

Contrato según SRS
Precondición: nombre != null .
Postcondición: El atributo nombre del DTO toma el valor del parámetro ingresado (this.nombre == nombre).

Tabla partición

Dato de entrada	Clase válida	Clase inválida
nombre	nombre!=NULL (1)	No aplica

Batería de pruebas

Número de prueba	Dato de entrada	Valor	Salida esperada	Clases que alcanza
1	nombre	”Juan”	ninguna	1

public void SetApellido(String apellido)

Tabla partición
Contrato según SRS
Precondición: apellido != null .

Postcondición: El atributo apellido del DTO toma el valor del parámetro ingresado (this.apellido == apellido).

Dato de entrada	Clase válida	Clase inválida
apellido	apellido!=NULL (1)	No aplica

Batería de pruebas

Número de prueba	Dato de entrada	Valor	Salida esperada	Clases que alcanza
1	apellido	”Casanovas”	ninguna	1

public void SetDni(String dni)

Contrato según SRS
Precondición: dni != null . Actúa como identificador único del asociado.
Postcondición: El atributo dni del DTO almacena la cadena asignada (this.dni == dni).

Tabla partición

Dato de entrada	Clase válida	Clase inválida
dni	dni!=NULL (1)	No aplica

Batería de pruebas

Número de prueba	Dato de entrada	Valor	Salida esperada	Clases que alcanza
1	dni	”12345678”	ninguna	1

public void SetCiudad(String ciudad)

Contrato según SRS
Precondición: ciudad != null .
Postcondición: El atributo ciudad del DTO toma el valor del parámetro ingresado (this.ciudad == ciudad).

Tabla partición

Dato de entrada	Clase válida	Clase inválida
ciudad	ciudad!=NULL (1)	No aplica

Batería de pruebas

Número de prueba	Dato de entrada	Valor	Salida esperada	Clases que alcanza
1	ciudad	”Mar del plata”	ninguna	1

public void SetCalle(String calle)

Contrato según SRS
Precondición: calle != null .
Postcondición: El atributo calle del DTO almacena la dirección provista (this.calle == calle).

Tabla partición

Dato de entrada	Clase válida	Clase inválida
calle	calle!=NULL (1)	No aplica

Batería de pruebas

Número de prueba	Dato de entrada	Valor	Salida esperada	Clases que alcanza
1	calle	”Av. Colon”	ninguna	1

public void SetTelefono(String telefono)

Contrato según SRS
Precondición: telefono != null .
Postcondición: El atributo telefono del DTO almacena el número de contacto (this.telefono == telefono).

Tabla partición

Dato de entrada	Clase válida	Clase inválida

telefono	telefono!=NULL (1)	No aplica

Batería de pruebas

Número de prueba	Dato de entrada	Valor	Salida esperada	Clases que alcanza
1	telefono	”2231234567”	ninguna	1

public void SetNumero(int numero)

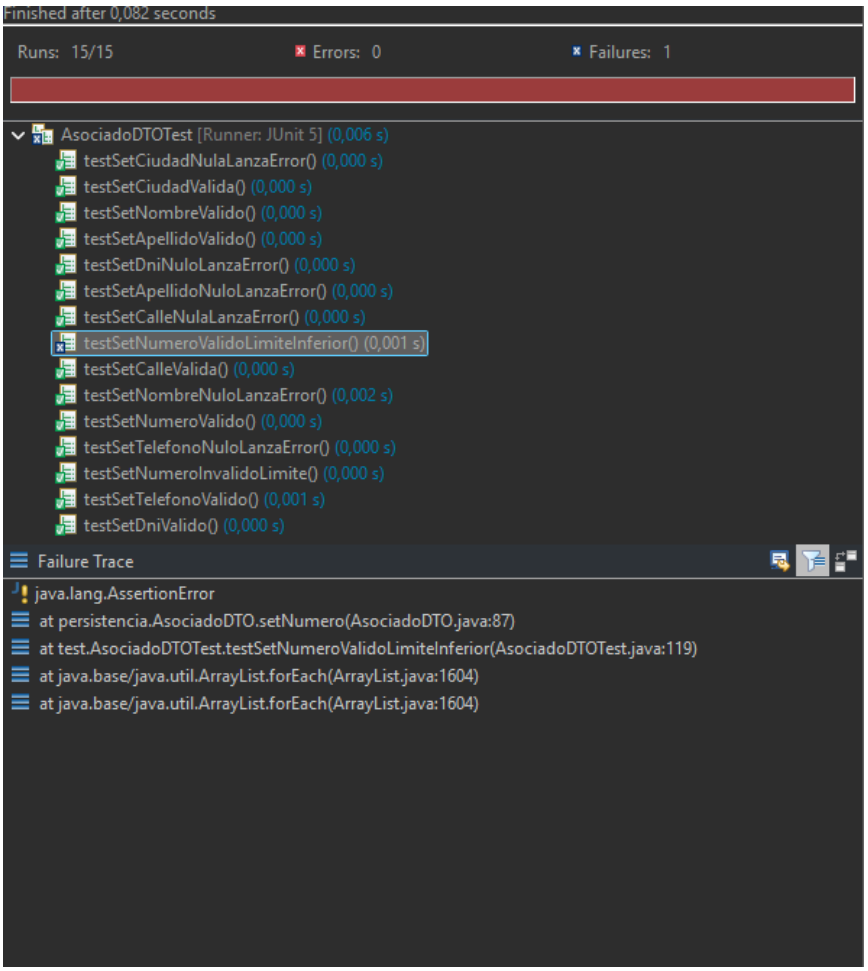
Contrato según SRS
Precondición: numero >= 0 (el número de calle o altura debe ser mayor o igual a 0).
Postcondición: El atributo numero del DTO almacena el valor ingresado (this.numero == numero).

Tabla partición

Dato de entrada	Clase válida	Clase inválida
numero	numero >= 0 (1)	numero < 0 (2)

Batería de pruebas

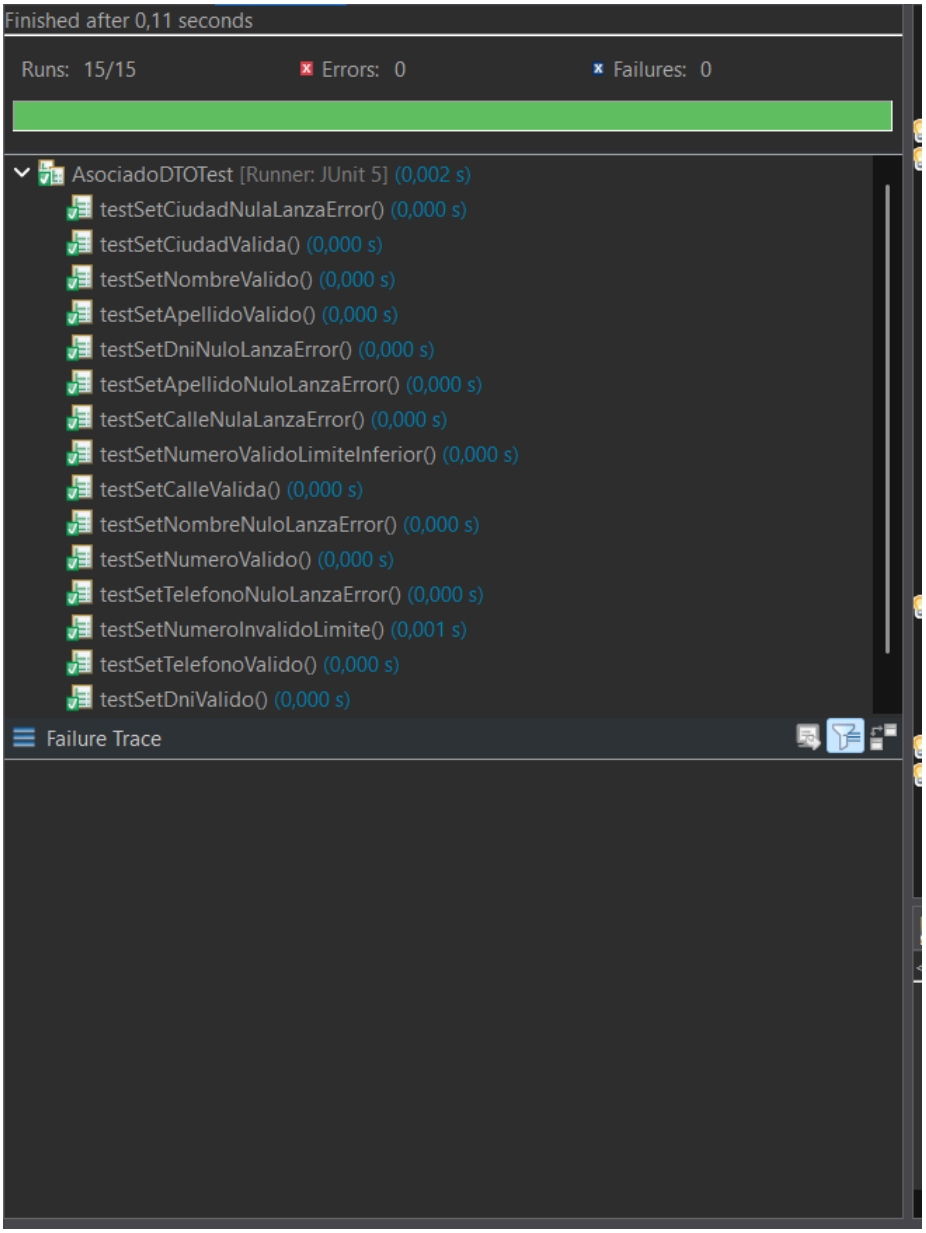
Número de prueba	Dato de entrada	Valor	Salida esperada	Clases que alcanza
1	numero	1	ninguna	1
2	numero	0	assert numero >= 0;	1
3	numero	-1	assert numero >= 0;	2



Resultados Obtenidos y Análisis de Inconsistencias

- Ejecución Inicial sobre el Código Base Provisto (Barra en Rojo):
  - Tests ejecutados: 15 | Exitosos: 14 | Fallos (Failures): 1 | Errores: 0.
  - Fallo registrado: testSetNumeroValidoLimiteInferior arrojó AssertionError no interceptado.
  - Causa técnica del defecto: El código original de AsociadoDTO define en la línea 79 assert numero > 0;. Al evaluar el valor límite válido 0, la aserción evalúa a false, interrumpiendo el flujo. Esto contradice el requerimiento del sistema que estipula que la altura o número debe ser mayor o igual a cero (número>=0).
  - Acción: Se adjunta captura de pantalla en rojo como evidencia de detección de errores.
- Corrección Implementada en AsociadoDTO.java:

```
public void setNumero(int numero) {
    assert numero >= 0;
    this.numero = numero;
}
```



Segunda Ejecución tras la Corrección (Barra en Verde):

- Tests ejecutados: 15 | Exitosos: 15 | Fallos: 0 | Errores: 0 (100% de éxito).
- Conclusión: Se verifica el correcto funcionamiento del 100% de los métodos de acceso y modificación, validando tanto la recuperación de datos válidos como la efectividad de las aserciones defensivas de integridad.
- Acción: Se adjunta captura de pantalla en verde certificando el funcionamiento correcto.

Apéndice B

Definición de escenarios

Nro. de escenario	Descripción del escenario
1	Base de Datos Inicializada con Datos Semilla: El motor MySQL se encuentra activo, la tabla Asociados fue recreada mediante iniciarBD() y poblada con los 20 registros iniciales predefinidos. El manejador Singleton mantiene una conexión JDBC abierta y válida.
2	Servicio de Base de Datos No Disponible: El servidor MySQL se encuentra apagado o con credenciales erróneas, imposibilitando el enlace JDBC con la aplicación.

Tablas de Partición y Baterías de Pruebas

Public static BaseDeDatos getInstancia()

(Escenario 1)
Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Estado de la instancia Singleton en memoria	Instancia ya creada previamente en memoria(1)	Instancia no creada previamente en memoria(2)
Estado del enlace JDBC	Conexión con MySQL activa y no nula (3)	Conexión nula o cerrada tras desconexión (4)

Batería de Pruebas

Tipo de clase	Invocación / Estado	Salida esperada	Clases cubiertas	Método @Test Asociado

Correcta	BaseDeDatos.getInstancia() sucesivo	Devuelve exactamente la misma referencia de memoria Conexión activa no nula.	1,3	testPatronSingleton()
Correcta	Primera llamada a getInstancia()	Crea la instancia única y abre la conexión con MySQL sin duplicar recursos.	2,3	testPatronSingleton()

Public void iniciarBD()

(Escenario 1 y 2)

Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Disponibilidad de conexión MySQL	Conexión activa con el motor relacional (1)	Motor MySQL apagado o sin red (2)
Estado previo de la tabla Asociados	Tabla existente con o sin datos previos (3)	-

Batería de Pruebas

Tipo de clase	Invocación / Estado	Salida esperada	Clases cubiertas	Método @Test Asociado
Correcta	bd.iniciarBD() bajo Escenario 1	Ejecuta script DDL (DROP / CREATE), recrea la tabla y puebla exactamente 20 asociados semilla.	1,3	testInicializacionYDatosSemilla()

Incorrecta	bd.iniciarBD() bajo Escenario 2	Dispara SQLException por fallo de enlace (CommunicationsException).	2	Control de conexión JDBC

Public void guardar(Asociado asociado)

(Escenario 1)
Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Objeto de Dominio asociado	Objeto no nulo con atributos consistentes (1)	Referencia nula (2) [null]
Columna DNI en tabla MySQL	DNI no existente en la base de datos (3)	DNI ya existente en la base de datos (viola UNIQUE) (4)

Batería de Pruebas

Tipo de clase	Invocación / Parámetros	Salida esperada	Clases cubiertas	Método @Test Asociado
Correcta	dao.guardar(nuevo) con DNI nuevo "99887766"	Se inserta la fila en MySQL; la cantidad total en BD pasa de 20 a 21.	1, 3	testGuardarAsociadoValido()
Incorrecta	dao.guardar(duplicado) con DNI "12345678" (Juan State)	MySQL rechaza la fila y dispara SQLException (Duplicate entry for key DNI).	1, 4	testGuardarAsociadoDniDuplicadoLanzaSQLException()
Incorrecta	dao.guardar(null)	Falla en precondición del DAO (assert != null), lanzando AssertionError sin tocar MySQL.	2	testGuardarAsociadoNuloLanzaError()

Correcta (Mapeo)	dao.guardar(grace) con datos no cruzados	Inserción exitosa; las 7 columnas se recuperan en su correspondencia biunívoca exacta.	1, 3	testMapeoCompletoAlGuardar()

Public ArrayList<AsociadoDTO> cargarConLimite(int limite)

(Escenario 1)
Tabla de Particiones
(Análisis de Valores Límite: precondición limite > 0)

Condición de entrada	Clases Válidas	Clases Inválidas
Parámetro limite (int)	Entero mayor a 0: • Límite inferior válido: limite = 1 (1) • Valor nominal: limite = 5 (2)	Entero menor o igual a 0: • Límite frontera inválido: limite = 0 (3) • Valor inválido exterior: limite = -1 (4)

Batería de Pruebas

Tipo de clase	Invocación / Parámetros	Salida esperada	Clases cubiertas	Método @Test Asociado
Correcta (Límite)	dao.cargarConLimite(1)	Retorna ArrayList<AsociadoDTO> con tamaño exactamente igual a 1.	1	testCargarConLimiteValido()
Incorrecta (Límite)	dao.cargarConLimite(0)	Violación de precondición: lanza AssertionError.	3	testCargarConLimiteInvalido()

Public AsociadoDTO cargar(int id)

(Escenario 1)
Tabla de Particiones
(Análisis de Valores Límite: precondición id > 0)

Condición de entrada	Clases Válidas	Clases Inválidas
Parámetro id (int)	Entero mayor a 0 existente en BD: • Límite inferior válido: id = 1 (1)	Entero menor o igual a 0: • Límite frontera inválido: id = 0 (2) • Valor negativo: id = -1 (3)
Existencia en la tabla	ID registrado en BD (4)	ID no registrado en BD (id > 20) (5)

Batería de Pruebas

Tipo de clase	Invocación / Parámetros	Salida esperada	Clases cubiertas	Método @Test Asociado
Correcta (Límite)	dao.cargar(1)	Retorna el AsociadoDTO de Juan State (DNI "12345678"). En código original falla con SQLException.	1, 4	testCargarIndividualPorId()
Incorrecta (Límite)	dao.cargar(0)	Violación de precondición: lanza AssertionError.	2	testCargarPorIdInvalido()

Public void actualizar(Asociado asociado)

(Escenario 1)
Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas

Entidad asociado recibida	Objeto no nulo con DNI existente en BD (1)	Objeto nulo (2)
Identificador unívoco (DNI)	DNI perteneciente al registro a modificar (3)	DNI inexistente en la tabla (4)

Batería de Pruebas

Tipo de clase	Invocación / Parámetros	Salida esperada	Clases cubiertas	Método @Test Asociado
Correcta	dao.actualizar(mariaModificada) (DNI "87654321", ID 2)	Modifica la calle y número de María en BD sin afectar otros registros. El código provisto falla por ID fijo.	1, 3	testActualizarAsociadoExistente()

Public void eliminar(String dni)

(Escenario 1)
Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Parámetro dni (String)	Cadena no nula y no vacía de asociado existente (1)	Cadena vacía "" (2) Referencia nula null (3)

Batería de Pruebas

Tipo de clase	Invocación / Parámetros	Salida esperada	Clases cubiertas	Método @Test Asociado

Correcta	dao.eliminar("12345678") (Juan State)	Elimina la fila de Juan State; el total en BD pasa de 20 a 19 registros.	1	testEliminarAsociadoPorDni()
Incorrecta	dao.eliminar("")	Violación de precondición: lanza AssertionError.	2	testEliminarConDniVacioLanzaError()
Incorrecta (Bug)	dao.eliminar(null)	Violación de contrato: debe lanzar AssertionError (el código original lo deja filtrar).	3	testEliminarConDniNuloLanzaError()

Resultados obtenidos

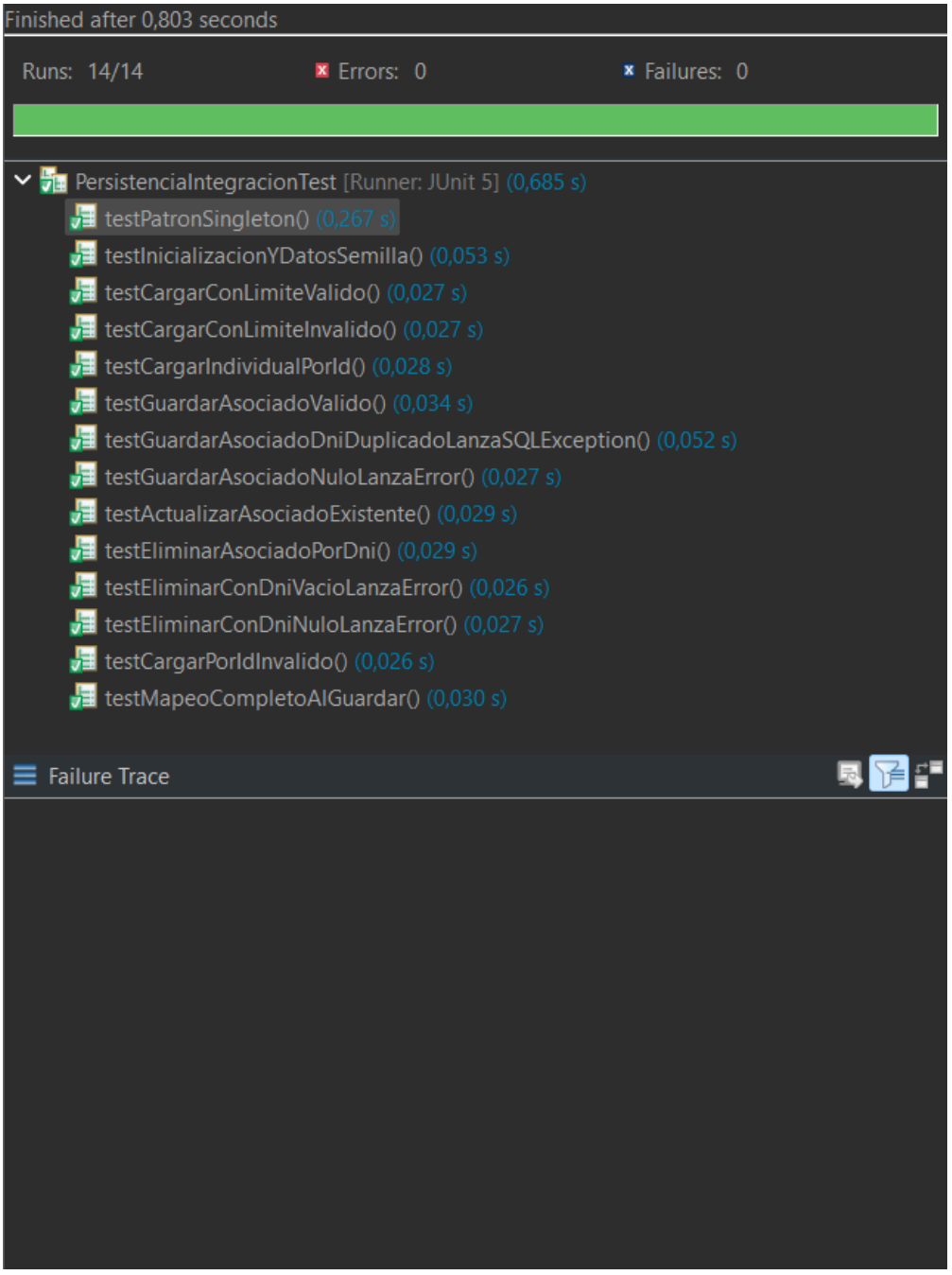
1. **Falta de avance del cursor en DataAccessObject.cargar(int id) (Detectado):**
    - *Código provisto:* Ejecuta la consulta SQL e inmediatamente intenta leer resultado.getInt("id") sin llamar antes a resultado.next().
    - *Impacto:* Dispara java.sql.SQLException: Before start of result set.
2. **Parámetro Hardcodeado en BaseDeDatos.ejecutarActualizacion (Detectado):**
    - *Código provisto:* stmt.setInt(8, 1);.
    - *Impacto:* Cualquier intento de actualizar a cualquier asociado sobreescribe forzosamente los datos del asociado cuyo id = 1, o falla por duplicación de clave única (DNI UNIQUE).
3. **Inconsistencia de Criterio de Actualización vs. SRS 3.3:**
    - *Código provisto:* La consulta SQL en DataAccessObject.actualizar finaliza en WHERE id = ?.

- *SRS 3.3:* Exige que las modificaciones y bajas se realicen *"basándose en criterios únicos como el número de documento (DNI)"*.

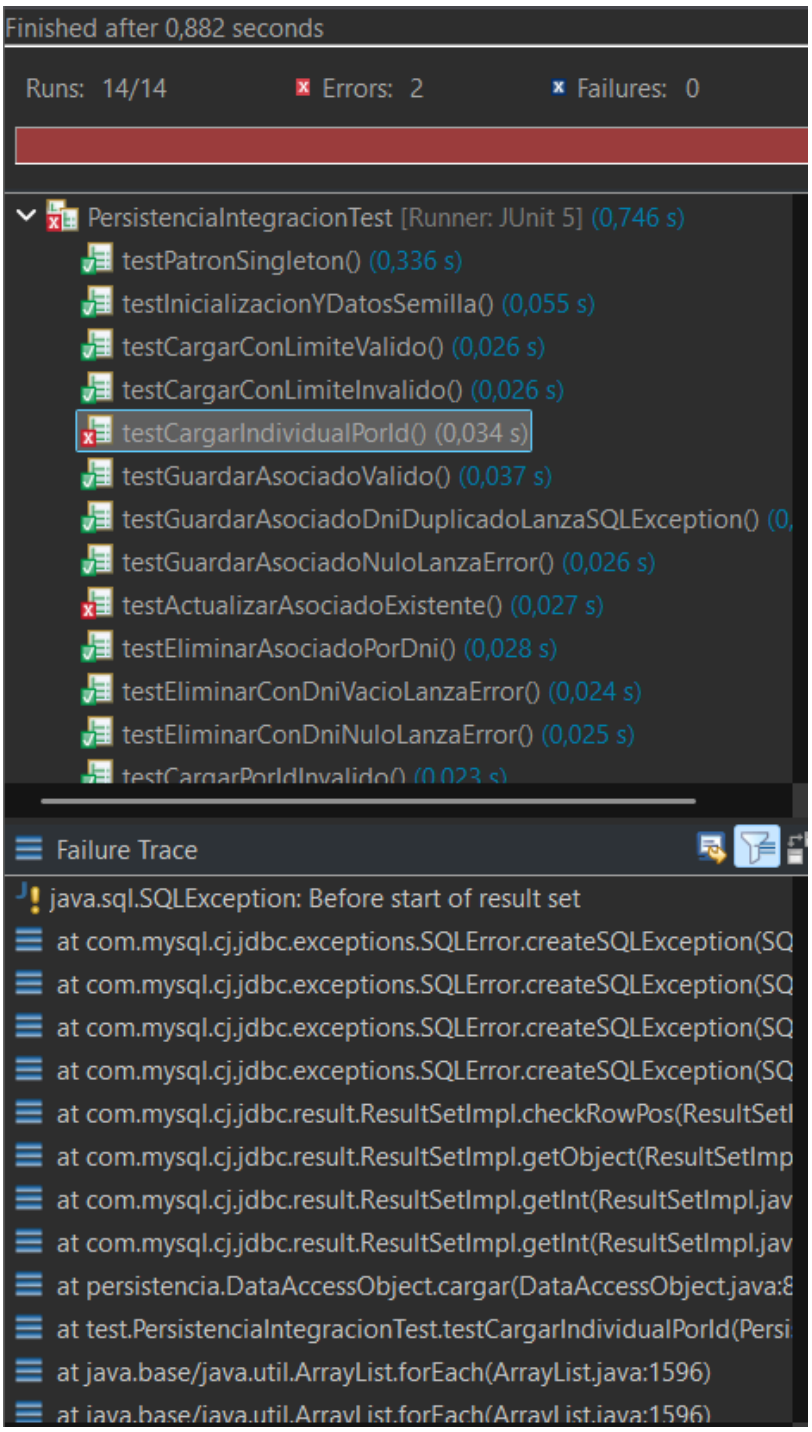
observación= Validación defectuosa en DataAccessObject.eliminar(String dni):

- *Código provisto:* assert !Objects.equals(dni, "") : "El DNI no puede ser nulo";.
- *Impacto:* la validación del DAO solo verifica que el DNI no sea una cadena vacía (""), pero no contempla el caso de que sea null. Por eso, si el valor es null, la validación del DAO no lo detecta y se ejecuta, aunque su mensaje de error diga "El DNI no puede ser nulo". De todas formas, el null no llega a la base de datos, porque BaseDeDatos.ejecutarDelete() tiene su propia validación (assert dni != null) y lo rechaza. Por este motivo, la prueba testEliminarConDniNuloLanzaError() pasó en verde.

Verificación de la conexión del patrón Singleton



TestCargarIndividualPorId()



El error `java.sql.SQLException: Before start of result set` se produce cuando el programa intenta obtener información de la base de datos, como un valor entero o una cadena de texto, sin haber posicionado previamente el cursor sobre un registro del resultado de la consulta.

En JDBC, al ejecutar una consulta y obtener un objeto `ResultSet`, el cursor se encuentra inicialmente antes de la primera fila. Por este motivo, es necesario invocar el método `.next()` para avanzar hacia el primer registro y poder acceder a los valores de sus columnas.

De acuerdo con el registro de errores de la consola, el problema se encuentra en el método `cargar()` de la clase `DataAccessObject`, específicamente en la línea 88. El fallo ocurre durante la ejecución de la prueba `testCargarIndividualPorId`, cuyo objetivo es recuperar un asociado a partir de su identificador único.

Para corregir este inconveniente, es necesario modificar el método `cargar()` del DAO, incorporando una condición `if (resultado.next())` antes de realizar la lectura de las columnas. De esta manera, se garantiza que el programa solamente intente acceder a los datos cuando la consulta haya devuelto un registro válido.

Este es el código que genera el error

```
public AsociadoDTO cargar(int id) throws SQLException {
    assert  id > 0 : "El id debe ser mayor que 0";
    String query = "SELECT * FROM Asociados WHERE id = ? ;";

    ResultSet resultado = bd.ejecutarConsulta(query, id);

    AsociadoDTO aDTO = null;
    aDTO = this.CrearDTO(resultado.getInt("id"), resultado.getString("nombre"), resultado.getString("apellido"), resultado.getString("dni")
           , resultado.getString("calle"), resultado.getInt("numero"), resultado.getString("ciudad"), resultado.getString("telefono"));
    resultado.close();
    return aDTO;
}
```

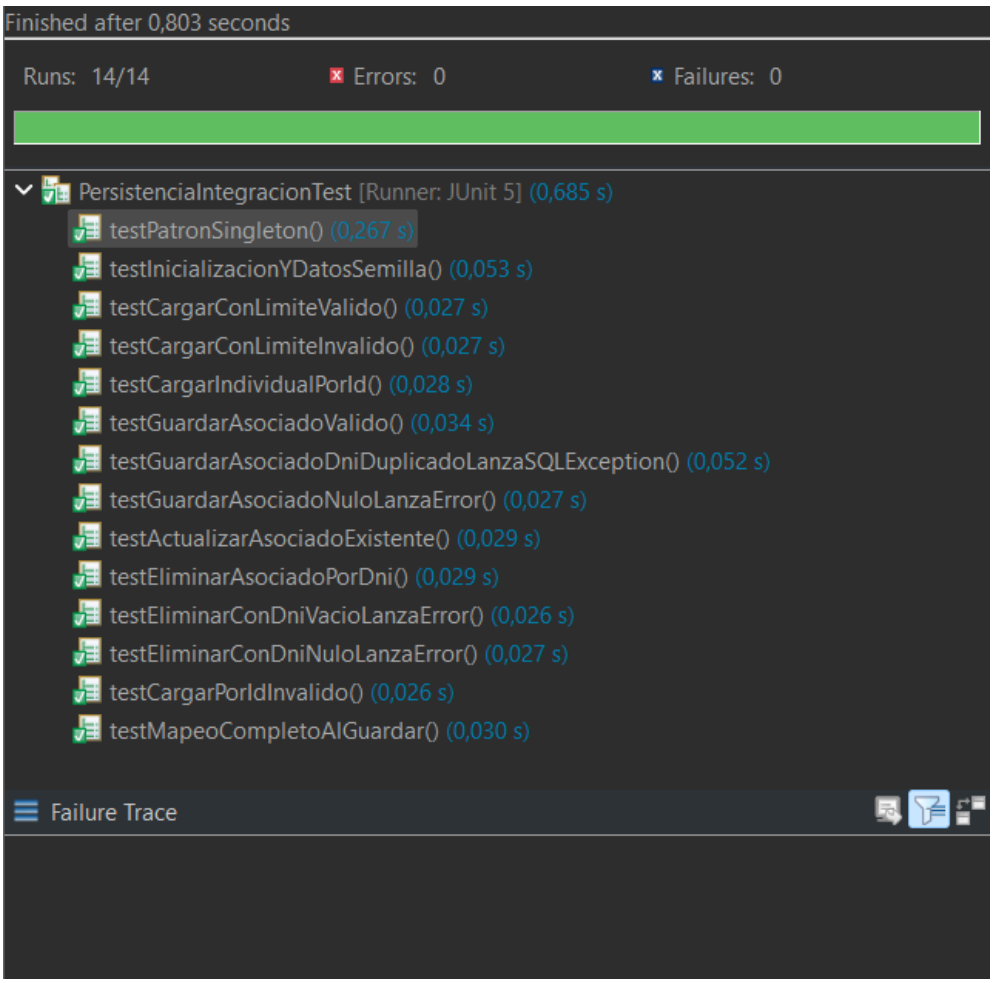
este es el código que planteamos como solución

```
public AsociadoDTO cargar(int id) throws SQLException {
    assert  id > 0 : "El id debe ser mayor que 0";
    String query = "SELECT * FROM Asociados WHERE id = ? ;";

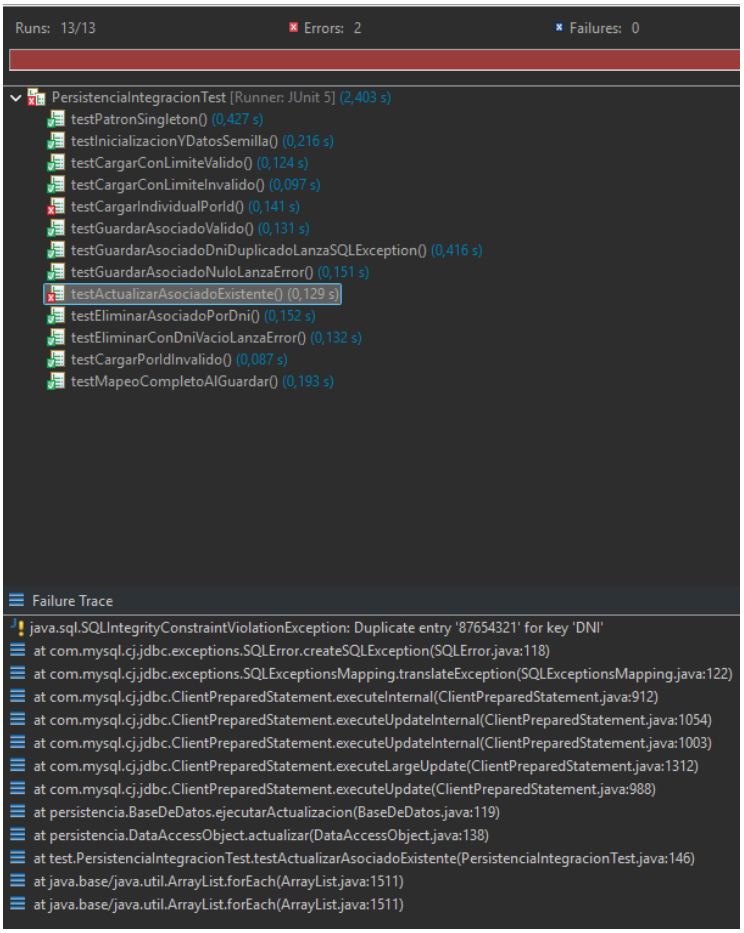
    ResultSet resultado = bd.ejecutarConsulta(query, id);

    if(resultado.next()) {
        AsociadoDTO aDTO = null;
        aDTO = this.CrearDTO(resultado.getInt("id"), resultado.getString("nombre"), resultado.getString("apellido"), resultado.getString("dni")
               , resultado.getString("calle"), resultado.getInt("numero"), resultado.getString("ciudad"), resultado.getString("telefono"));
        resultado.close();
        return aDTO;
    }
    else
        return null;

}
```


Finished after 0,803 seconds
Runs: 14/14
Errors: 0
Failures: 0
PersistenciaIntegracionTest [Runner: JUnit 5] (0,685 s)
testPatronSingleton() (0,267 s)
testInicializacionYDatosSemilla() (0,053 s)
testCargarConLimiteValido() (0,027 s)
testCargarConLimiteInvalido() (0,027 s)
testCargarIndividualPorId() (0,028 s)
testGuardarAsociadoValido() (0,034 s)
testGuardarAsociadoDniDuplicadoLanzaSQLException() (0,052 s)
testGuardarAsociadoNuloLanzaError() (0,027 s)
testActualizarAsociadoExistente() (0,029 s)
testEliminarAsociadoPorDni() (0,029 s)
testEliminarConDniVacioLanzaError() (0,026 s)
testEliminarConDniNuloLanzaError() (0,027 s)
testCargarPorIdInvalido() (0,026 s)
testMapeoCompletoAlGuardar() (0,030 s)
Failure Trace

Resultado de la prueba del crud



Error: Descripción del Error

Durante la ejecución de las pruebas orientadas a validar la operación de actualización de registros de asociados, la prueba testActualizarAsociadoExistente fue interrumpida por la excepción java.sql.SQLIntegrityConstraintViolationException: Duplicate entry '87654321' for key 'DNI'. El error se originó en el componente BaseDeDatos, dentro del método ejecutarActualizacion, invocado por el DataAccessObject (DAO) al actualizar un asociado.

El problema se encontraba en la asignación de parámetros de la sentencia SQL utilizada para actualizar los datos:

sentencia = “ UPDATE Asociados SET nombre = ?, apellido = ?, dni = ?, calle = ?, numero = ?, ciudad = ?, telefono = ? WHERE id = ? ”.
El último parámetro de esta consulta determina qué registro debe modificarse. Sin embargo, en el método `ejecutarActualizacion` se le asignaba un valor constante mediante la instrucción `stmt.setInt(8, 1)`. Esto provocaba que todas las operaciones de actualización se aplicarán sobre el asociado cuyo identificador era `1`, independientemente del registro que se quisiera modificar.

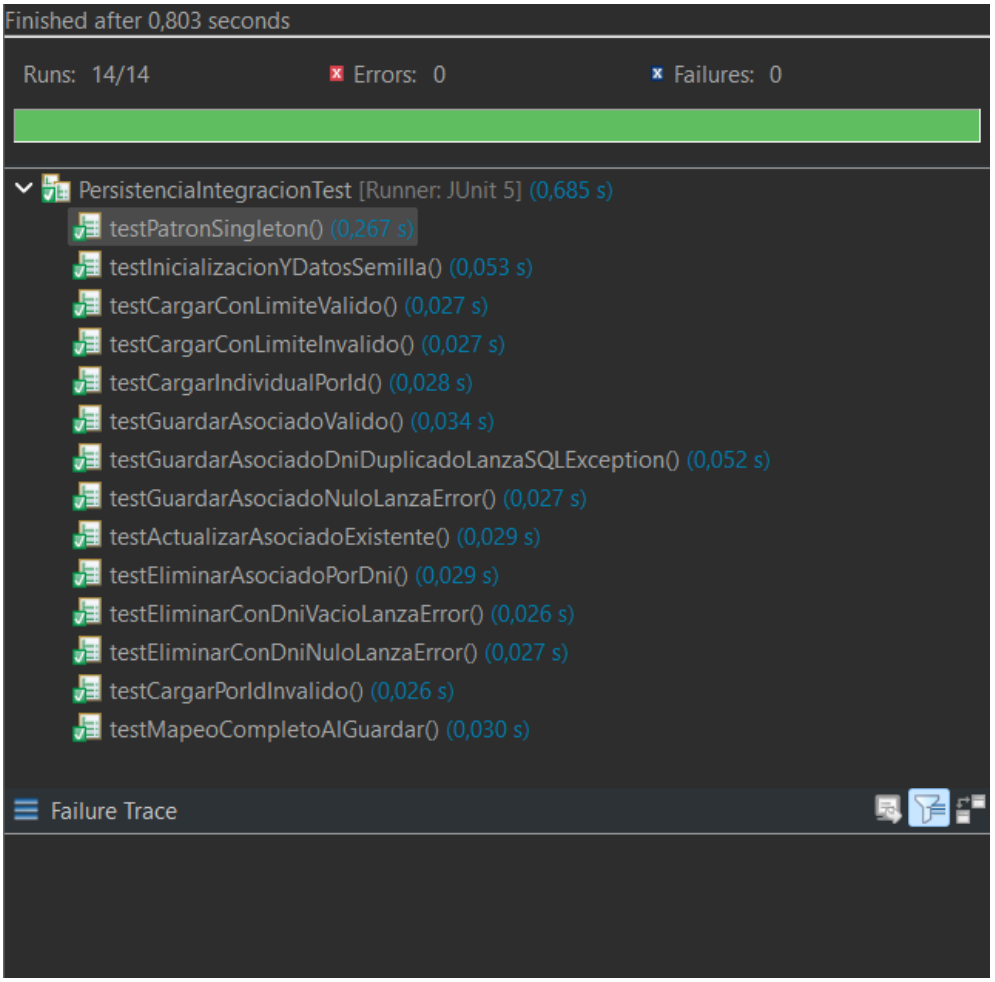
En la prueba se intentó modificar a la asociada con DNI 87654321 (id 2). La sentencia intentó escribir ese DNI sobre el registro con id 1, perteneciente a otro asociado.

Dado que la columna DNI posee una restricción UNIQUE y el valor ya existía en la tabla, el motor MySQL rechazó la operación y lanzó la excepción.

Solución Propuesta

Para subsanar este comportamiento, se propone modificar el criterio de selección de la sentencia de actualización, reemplazando la condición WHERE id = ? por WHERE dni = ? en el DataAccessObject.

En el componente BaseDeDatos, el valor fijo stmt.setInt(8, 1) se reemplaza por stmt.setString(8, aDTO.getDni()), de modo que el parámetro sea asignado dinámicamente con el DNI del asociado recibido.



Apéndice C

Definición Formal de Escenarios

Nro de Escenario	Descripción del Escenario
1	Ambulancia desocupada, simulación activa y estadoMock inyectado. El mock no tiene una excepción configurada. Se invoca una operación para verificar su delegación al estado.
2	Ambulancia desocupada, simulación activa y estadoMock inyectado. SolicitudMantenimiento() del mock está configurado para lanzar IllegalStateException. Se evalúa la propagación de esa excepción.
3	Ambulancia con estadoMock inyectado y simulación activa mediante setSimulacionActiva(true). Se ejecuta new RetornoAutomatico(ambulancia) en un Thread independiente y se verifica la interacción con el estado simulado.
4	Ambulancia ocupada mediante setOcupado(true), simulación activa y estadoMock inyectado. Un hilo solicita atenderDomicilio con un asociado mock y alcanza el estado WAITING. El hilo del test invoca retornoAutomatico con una rutina no nula y comprueba el avance de la solicitud pendiente.

Tablas de Particiones y Baterías de Pruebas

Public void solicitarMantenimiento(Operario o)

(Escenario 1)
Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Parámetro o (Operario)	Instancia de operario no nula (1)	Referencia nula null (2) (contrato)

Batería de Pruebas

Tipo de clase	Valores de entrada	Salida esperada	Clases cubiertas	Escenario
Correcta	o = mock(Operario.class). Estado sin excepción configurada.	Se invoca estadoMock.SolicitudMantenimiento() exactamente una vez y no se producen otras interacciones con estadoMock.	1	1
Correcta	o = mock(Operario.class). Estado configurado con doThrow.	Ambulancia propaga la excepción hacia el llamador sin atraparla silenciosamente.	1	2

La excepción programada no viola la precondición o != null.

Public void atenderDomicilio(Asociado a)

Escenario 1. Precondición: asociado != null. La ambulancia libre y la simulación activa son condiciones fijas de este escenario.
Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Parámetro a (Asociado)	Instancia de asociado no nula (1)	Referencia nula null (2) Sin caso implementado

Batería de Pruebas

Tipo de clase	Parámetros	Salida esperada	Clases cubiertas	Escenario
Correcta	asociado = mock(Asociado.class). getNombre() devuelve "Juan". Se invoca atenderDomicilio(asociado).	Delega a estadoMock.SolicitudDeAtencionDomicilio() 1 vez.	1	1

Public void trasladarALaClinica(Asociado a)

Escenario 1. Precondición: asociado != null. La ambulancia comienza libre y la simulación activa.

Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Parámetro a (Asociado)	Instancia de asociado no nula (1)	Referencia nula null (2) Sin caso implementado

Batería de Pruebas

Tipo de clase	Parámetros	Salida esperada	Clases cubiertas	Escenario
Correcta	asociado = mock(Asociado.class). Se invoca trasladarALaClinica(asociado). No se configura getNombre().	Delega a estadoMock.SolicitudDeTraslado() exactamente 1 vez.	1	1

Public void regresarSinPaciente()

Escenario 1. Método sin parámetros, invocado sobre la instancia preparada con estadoMock.

Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
No aplica	No aplica particionar parámetros. Se define una invocación en el escenario 1.	No aplica. No recibe parámetros explícitos.

Batería de Pruebas

Tipo de clase	Parámetros	Salida esperada	Clases cubiertas	Escenario
Correcta	Sin parámetros. Se invoca ambulancia.regresarSinPaciente().	Delega a estadoMock.RetornoClinica() exactamente 1 vez, sin otras interacciones con ese mock.	No aplica	1

Rutina Concurrente: RetornoAutomatico.run()

Escenario 3: run() no recibe parámetros. La ambulancia no nula se proporciona al constructor de RetornoAutomatico.

Tabla de Particiones

Condición de entrada	Clases Válidas	Clases Inválidas
No aplica	No aplica particionar parámetros. Se define una invocación en el escenario 3.	No aplica. No recibe parámetros explícitos.

Batería de Pruebas

Tipo de clase	Valores de entrada	Salida esperada	Clases cubiertas	Escenario
Correcta	new RetornoAutomatico(ambulancia).setSimulacionActiva(true).	El test exige al menos una invocación de estadoMock.RetornoClinica() al finalizar timeout(11000).	No aplica	3

	Se inicia el Thread y se verifica con timeout(11000)			

retornoAutomatico(RetornoAutomatico r)

Escenario 4: Se usa r = new RetornoAutomatico(ambulancia). Otro hilo llama a atenderDomicilio(a), donde a = mock(Asociado.class). La ocupación, la simulación activa y la existencia de un solicitante en espera son condiciones del escenario.

Tabla de particiones

Condición de entrada	Clases Válidas	Clases Inválidas
Parámetro r (RetornoAutomatico)	(1) Rutina no nula asociada a la ambulancia. Representante: new RetornoAutomatico(ambulancia).	No se especifica una precondición que permita clasificar r = null como inválido. Ese valor no se prueba.

Batería de Pruebas

Tipo de clase	Valores de entrada	Salida esperada	Clases cubiertas	Escenario
Correcta (Secuencial)	r = new RetornoAutomatico(ambulancia). ocupado = true.	El solicitante alcanza WAITING. Tras el retorno, el test exige una llamada a SolicitudDeAtencionDomicilio() sobre estadoMock dentro de 1.000 ms mediante verify(timeout(1000)).	1	4

Resultados Obtenidos y Análisis de Inconsistencias

Resultados obtenidos

1. Condición invertida en RetornoAutomatico.run() (Detectado):

Código provisto: el bucle de la rutina se define como while (!ambulancia.isSimulacionActiva()).

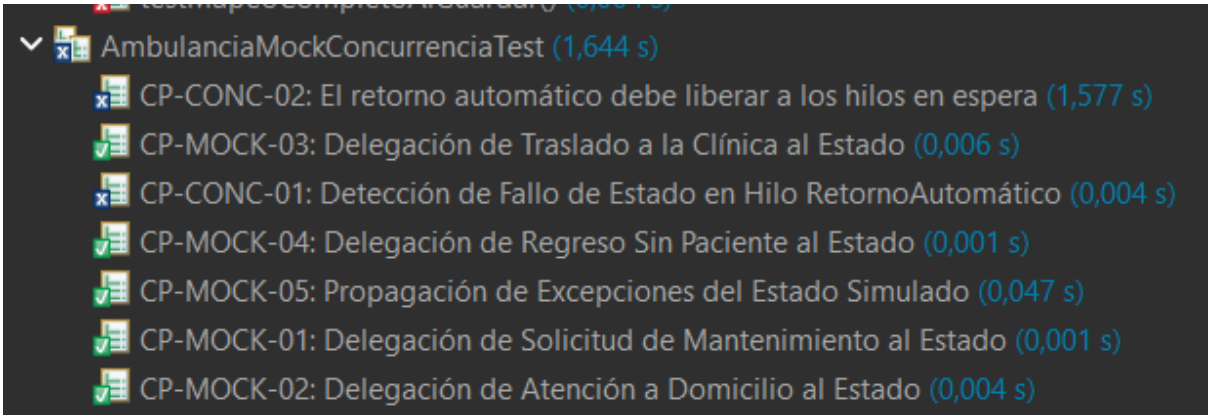
Impacto: con la simulación activa, el hilo finaliza inmediatamente sin ejecutar nunca el retorno automático de la ambulancia.

2. Falta de liberación de la ambulancia en Ambulancia.retornoAutomatico() (Detectado):

Código provisto: el método ejecuta estadoActual.RetornoClinica() y notifyAll(), pero no invoca setOcupado(false).

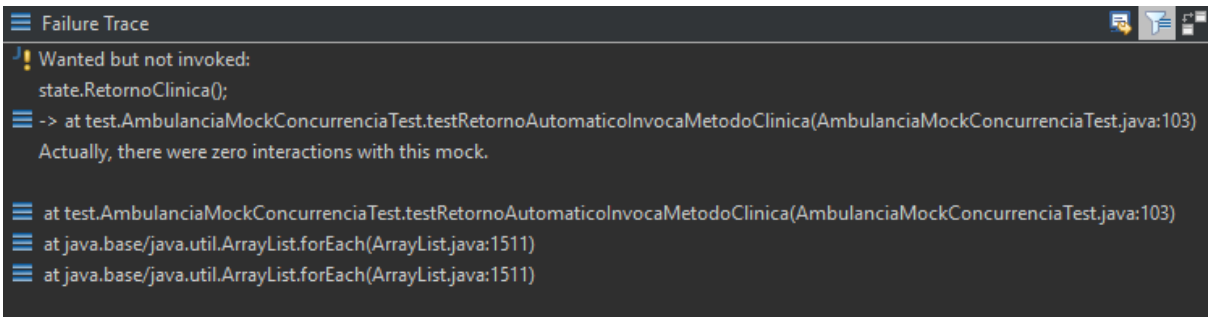
Impacto: los hilos que esperan la ambulancia se despiertan, la encuentran todavía ocupada y vuelven a esperar. Las solicitudes pendientes quedan bloqueadas indefinidamente.

Resultado de las pruebas con mocks



Las cinco pruebas con mocks (CP-MOCK-01 a CP-MOCK-05) finalizaron correctamente. Se verificó que Ambulancia delega cada operación en el método correspondiente de la interfaz State, sin producir interacciones adicionales, y que propaga hacia el llamador las excepciones generadas por el estado sin atraparlas. Las dos pruebas de concurrencia fallaron, ya que detectan defectos que solo se manifiestan al ejecutar hilos reales.

testRetornoAutomaticoInvocaMetodoClinica() – CP-CONC-01



Durante la ejecución de la prueba CP-CONC-01, Mockito informó el fallo "Wanted but not invoked: state.RetornoClinica()" en la línea 103 del test, indicando además "Actually, there

were zero interactions with this mock". Esto significa que el hilo RetornoAutomatico finalizó sin invocar ningún método del estado de la ambulancia.

La prueba activa la simulación, inicia la rutina RetornoAutomatico en un hilo independiente y verifica que, en algún momento, se solicite el retorno de la ambulancia a la clínica. Sin embargo, el bucle de la rutina tiene la condición invertida: se ejecuta mientras la simulación NO está activa. Como la simulación está activa, la condición resulta falsa desde el inicio y el hilo termina inmediatamente.

Este es el código que genera el error

```java
@Override
public void run() {
    while (!ambulancia.isSimulacionActiva()) {
        try {
            Thread.sleep(INTERVALO_RETORNO);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return; // Salir limpiamente si el hilo es interrumpido
        }

        // Verificar nuevamente después del sleep
        if (!ambulancia.isSimulacionActiva()) {
            break;
        }

        // Intentar retorno automático
        // El método retornoAutomatico en Ambulancia debe verificar
        // si es apropiado hacer el retorno según el estado actual
        ambulancia.retornoAutomatico(this);
    }
}
```

Solución Propuesta

Para arreglar este error, se propone corregir la condición del bucle en RetornoAutomatico.run(), reemplazando while (!ambulancia.isSimulacionActiva()) por while (ambulancia.isSimulacionActiva()), de modo que la rutina se ejecute periódicamente mientras la simulación esté activa.

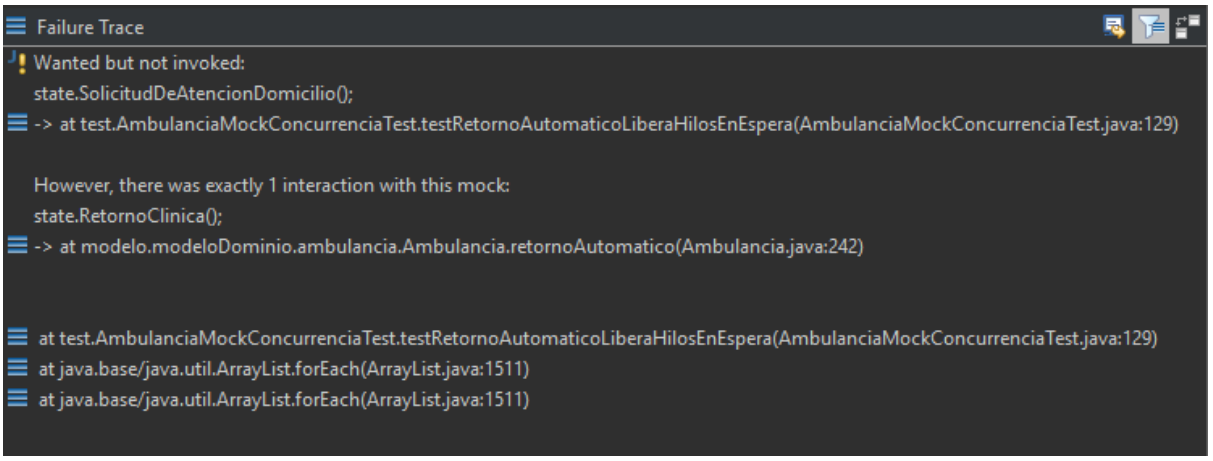
Este es el código que planteamos como solución

```java
@Override
public void run() {
    while (ambulancia.isSimulacionActiva()) {
        try {
            Thread.sleep(INTERVALO_RETORNO);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return; // Salir limpiamente si el hilo es interrumpido
        }

        // Verificar nuevamente después del sleep
        if (!ambulancia.isSimulacionActiva()) {
            break;
        }

        // Intentar retorno automático
        // El método retornoAutomatico en Ambulancia debe verificar
        // si es apropiado hacer el retorno según el estado actual
        ambulancia.retornoAutomatico(this);
    }
}
```

testRetornoAutomaticoLiberaHilosEnEspera() – CP-CONC-02



Error: Descripción del Error

Durante la ejecución de la prueba CP-CONC-02, Mockito informó el fallo "Wanted but not invoked: state.SolicitudDeAtencionDomicilio()" en la línea 125 del test, luego de esperar 1.000 ms. Además, indicó que hubo "exactly 1 interaction with this mock: state.RetornoClinica()", originada en Ambulancia.retornoAutomatico (Ambulancia.java:242). Es decir, el retorno automático se ejecutó y delegó correctamente en el estado, pero el hilo que solicitaba atención nunca fue atendido.

La prueba marca la ambulancia como ocupada y lanza un hilo que solicita atención a domicilio. Como la ambulancia está ocupada, el hilo queda esperando (estado WAITING), lo cual la prueba verifica antes de continuar. Luego se invoca retornoAutomatico(), que debería liberar la ambulancia y permitir que el hilo sea atendido.

El problema se encuentra en el método retornoAutomatico() de la clase Ambulancia: despierta a los hilos en espera mediante notifyAll(), pero no marca la ambulancia como libre. En consecuencia, el hilo solicitante despierta, vuelve a evaluar la condición while (isOcupado() && isSimulacionActiva()), encuentra la ambulancia todavía ocupada y vuelve a esperar, quedando bloqueado indefinidamente. A diferencia de este método, regresarSinPaciente() sí ejecuta setOcupado(false) antes de notifyAll(), lo que muestra que el comportamiento esperado ya existe en otro método de la misma clase.

Este es el código que genera el error

```
public synchronized void retornoAutomatico(RetornoAutomatico retornoAutomatico) {
    estadoActual.RetornoClinica();
    notifyAll();
}
```

Solución Propuesta

Para arreglar este problema, se propone incorporar la instrucción setOcupado(false) en el método retornoAutomatico(), antes de notifyAll(), de modo que los hilos en espera encuentren la ambulancia disponible al despertar.

Este es el código que planteamos como solución

```
public synchronized void retornoAutomatico(RetornoAutomatico retornoAutomatico) {
    estadoActual.RetornoClinica();
    this.setOcupado(false);
    notifyAll();
}
```

Conclusiones generales

La estrategia de testing aplicada permitió verificar el funcionamiento de la aplicación en tres niveles de integración creciente, y cada uno de ellos detectó defectos que los demás no habrían encontrado.

En las pruebas unitarias de AsociadoDTO, el análisis de valores límite permitió detectar que setNumero() rechazaba el valor 0, a pesar de que el contrato lo define como válido. Este error no se habría encontrado probando únicamente valores nominales, lo que muestra la importancia de elegir los casos de prueba en las fronteras de cada partición.

En las pruebas de integración contra MySQL se detectaron dos errores que solo aparecen al comunicarse con una base de datos real: la falta de un llamado a next() en DataAccessObject.cargar() y el parámetro fijo en BaseDeDatos.ejecutarActualizacion(), que hacía que toda modificación se aplicara sobre el mismo registro. Además, se identificó una inconsistencia con el SRS, ya que actualizar() busca el registro por id y no por DNI. Estas

pruebas también confirmaron el correcto funcionamiento del patrón Singleton y de la carga de los datos semilla.

En las pruebas de la clase Ambulancia, el uso de Mockito permitió aislarla de las implementaciones concretas de sus estados y verificar que delega correctamente cada operación en la interfaz State. Sin embargo, los dos defectos encontrados en este punto solo se manifestaron al ejecutar hilos reales: la condición invertida del bucle en RetornoAutomatico.run(), que impedía que la rutina se ejecutara, y la falta de setOcupado(false) en Ambulancia.retornoAutomatico(), que dejaba bloqueados a los hilos en espera.

En conjunto, el trabajo permitió detectar y corregir cinco defectos del código provisto. Una vez aplicadas las correcciones, todas las pruebas finalizaron correctamente, lo que confirma que los componentes evaluados cumplen con lo especificado.